Pattern Recognition 167 (2025) 111721



Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/pr





SVD-KD: SVD-based hidden layer feature extraction for Knowledge distillation

Jianhua Zhang [a,b], Yi Gao [a], Mian Zhou [c], Ruyu Liu [d,e,*], Xu Cheng [a], Saša V. Nikolić [f], Shengyong Chen [a]

[a] School of Computer Science and Engineering, Tianjin University of Technology, Tianjin, 300384, China
[b] College of Artificial Intelligence and Automation, Hohai University, Changzhou, 213000, China
[c] AI and Advanced Computing, XJTLU Entrepreneur College (Taicang), Xi'an JiaoTong-Livepool University, Suzhou, China
[d] School of Information Science and Technology, Hangzhou Normal University, Hangzhou, 311121, China
[e] Department of Technology, Management and Economics, Technical University of Denmark, Lyngby, 2800, Denmark
[f] Department of Electronics, Faculty of Electronic Engineering, University of Nis, Nis, Serbia

ARTICLE INFO

Dataset link: https://www.cs.toronto.edu/~kriz/cifar.html, https://image-net.org/index.php

Keywords:
Knowledge distillation
Singular value decomposition
Deep learning
Computer vision
Model compression

ABSTRACT

Recent advancement of Knowledge distillation (KD) is to extract and transfer middle-layer knowledge of teacher models to student models, which is better than original KDs which only transfer the last layer of knowledge. However, the middle-layer knowledge commonly appears as a high-dimensional tensor, which is more difficult to transfer than the one-dimensional knowledge in the last layer. Moreover, when there are significant differences between the teachers and students in terms of model parameter capabilities and model structures, the middle layers of teacher and student models differ in dimensionality and structure, which further increase the learning difficulty for student models. To solve these problems, we propose a novel knowledge extraction module to transform the high-dimensional tensor tensor-based knowledge in middle layers to one-dimensional knowledge based on singular value decomposition. Thus, the knowledge at the middle layers of teacher models can be effectively extracted and simplified, and extremely facilitate the learning of student models, even though the structure and parameter capacity between the teacher network and the student network are very different. To help the students learn the knowledge from the middle layers of teachers as accurately as possible, we also propose a novel loss function that can constrain values of the one-dimension knowledge learned by the student model to be as close as possible to that extracted from the teacher model, thus improving the learning efficiency of the student model. We have conducted extensive experiments on three major datasets (CIFAR-10, CIFAR-100, and ImageNet 1K), and the results of our method demonstrate superior performance by comparing with the state-of-the-art methods.

1. Introduction

In pattern recognition tasks, deep neural network models can significantly enhance recognition accuracy. However, as the model size and network depth increase, these models become challenging to deploy on resource-constrained edge devices. Therefore, before applying complex deep learning models to practical industrial scenarios (such as bionic robots [1] or fault detection [2]), model compression becomes crucial.

Knowledge Distillation (KD) [3], as a flexible and effective model compression technique, has been proven to address this issue effectively by transferring knowledge from large models to smaller ones, ensuring high performance while adapting to the computational limitations of edge devices. KD aims to pass the knowledge from a well-trained large network (called the teacher model) to a smaller network (called the student model) to enhance the performance of the student model. Through training, the student model attempts to achieve the ability of the teacher model as much as possible and eventually substitutes the teacher model to reduce the model size.

The initial proposed KD methods transferred the vector output by the last layer of teacher models as knowledge to student models [3]. Students learn the knowledge of teachers by closely fitting this vector as the probability output of the whole student model. However, the output of the last layer cannot represent the knowledge of the whole teacher model, along with the teacher model being deeper and deeper.

* Corresponding author at: School of Information Science and Technology, Hangzhou Normal University, Hangzhou, 311121, China.
E-mail addresses: zjh@ieee.org (J. Zhang), gaoyi01020304@stud.tjut.edu.cn (Y. Gao), zhoumian@tjut.edu.cn (M. Zhou), ruyu.liu@ieee.org (R. Liu), xu.cheng@ieee.org (X. Cheng), sasa.nikolic@elfak.ni.ac.rs (S.V. Nikolić), sy@ieee.org (S. Chen).

https://doi.org/10.1016/j.patcog.2025.111721
Received 8 March 2024; Received in revised form 15 January 2025; Accepted 16 April 2025
Available online 2 May 2025
0031-3203/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Thus, relying on the last layer alone for knowledge transfer makes it difficult for students to effectively learn from teachers [4,5]. Bengio et al. proposed to transfer the features at middle layers of teachers as knowledge to students so that students can leverage these features as additional supervision to improve their accuracy [6]. Many methods follow and improve this idea and achieve considerable progress [7,8]. The latest studies [9,10] aim to achieve better KD performance by establishing improved relational loss functions. However, these methods remain two main challenges that have yet to be addressed [11,12].

The first challenge is that the middle layer features are typically high-dimensional tensors. It is a complex problem to ensure that the student network accurately fits middle-layer values as closely as possible to those of the teacher network. Although some matrix mapping approaches can help students fit these values well [10], they diminish the independence in each channel in the middle layers of the teacher [13]. While some improvements have been proposed to make the middle layer of students, approximate that of teachers by finding relationships between them, the information embedded in the middle of a teacher network will be lost [14,15]. In most cases, these relationships often cannot allow the student network to accurately fit the middle layer of the teacher network.

The second challenge is that the middle-layer knowledge of the teacher and student networks, with very different structures and parameters, has inconsistent dimensions of HEIGHT, WIDTH and CHANNEL. Although the alignment can be achieved by mapping, it will additionally increase the training difficulty of the student network because an additional mapping relation matrix needs to be learned [16]. While some methods [17,18] have achieved this by introducing an intermediate network as a bridge, they still require a two-step operation and do not eliminate the complexity. There are also methods to prune the middle layer from the teacher network to align the student network, but again it is a two-step operation with high complexity [7,19].

To address these two challenges, we propose a simple but efficient knowledge extraction module for the middle layer, which utilizes Singular Value Decomposition (SVD) [20] to simplify the high-dimensional knowledge into one-dimensional (1D) knowledge. SVD decomposes a two-dimensional matrix into three matrices: the left singular vector matrix (U), the singular value matrix (S), and the right singular vector matrix (V). This decomposition captures the most important features in the data, where the singular values represent the importance of each feature. By selecting a subset of the singular values, we can reduce the dimensionality of the matrix while retaining the most critical information without losing the core details. Finally, the student network can learn the knowledge of the teacher network effectively by fitting the 1D singular values decomposed by SVD. Thus, the first challenge can be resolved effectively.

We can flexibly apply SVD to decompose the feature tensors in middle layers in different dimensions, such as the feature matrix on HEIGHT and WIDTH dimensions or CHANNEL dimension, to obtain a 1D vector of singular values. Because these singular values can be ordered from largest to smallest after SVD decomposition, many zeros are at their tails. Ignoring these zeros does not cause information loss, and the alignment between the middle layers of teacher and student networks can be achieved by ignoring those zeros in the middle layer of the teacher network. Thus, the second challenge can be resolved.

In addition, our method can further simplify the knowledge of the teacher network by ignoring the information corresponding to the small singular values decomposed from the middle layer of the teacher network, and consequently improve the learning efficiency of the student network.

We also propose a new loss function based on the KL divergence [3,5], which is different from the original KL divergence and will not only constrain the distribution of the student network to be similar to that of the teacher network in terms of shape but more importantly, it also ensures that the numerical values of two distributions are as close as possible. Thus, the student can learn from the teacher more accurately, and improve their final performance.

Although Lee et al. [21] proposed to use SVD in their KD framework, it is significantly different from our method. It focuses on the semi-supervised learning tasks, where the SVD method is only used to decompose the information of a specific network layer, while we use SVD to extract, simplify, and unify the middle-layer knowledge of complex teacher networks and improve the knowledge transfer throughout the KD process.

Our main contributions are as follows:

- We propose a simple but novel module that can efficiently extract the knowledge in middle layers of the teacher network by SVD. This module can transfer the multi-dimensional knowledge into the 1D one, and consequently improve the learning efficiency of student networks.
- The proposed knowledge extraction module can align the middle layers of teacher and student networks, even if their structures and parameter capacities have significant differences. Thus, simple student networks can efficiently learn from complex teacher networks.
- Our knowledge extraction module can also arrange the order of importance of knowledge in middle layers according to the characteristics of SVD, and consequently omit those unimportant knowledge corresponding to the small singular values to simplify the knowledge transfer.
- We design a novel loss function by adding a regularization to an original KL loss, through which the student network can learn from the teacher network more accurately, and consequently improve its performance.
- We have conducted extensive experiments on several popular benchmark datasets, and results demonstrate satisfactory performance by comparing with many state-of-the-art methods.

2. Related work

KD is one of the more flexible and excellent model compression methods, which was first proposed by Hinton et al. [3]. Subsequently, numerous KD methods have been derived [22,23]. The initial KD idea was to design smaller networks that are easy to deploy as student models and let more excellent teacher models with mature and large networks guide the training of the student network. To realize this idea, Hinton et al. [3] transfer the output of the last layer of the teacher network, in other words, the probability distribution derived from each category, as knowledge for the student network to learn. Since then, many related works [22,24] have followed this idea of transferring only the last-layer knowledge without introducing other additional knowledge. It confirms that the method of passing only the last-layer knowledge is effective, but it is not effective enough for difficult and large models.

The initial proposal to introduce additional supervision in the middle layers of a model aims at improving the learning efficiency of the student network. Fitnet [6] is the first work to extract information from the middle layers. It utilizes a matrix to map the information from the teacher network's middle layers to the same size as the corresponding layers in the student network and then carries out knowledge transfer. This approach inevitably disrupts the correspondence of information in the channel during the matrix mapping process, making it more challenging for the student network to learn. Later, as network models became deeper, traditional KD methods are no longer effective in enhancing the performance of student models. Therefore, methods involving additional supervision begin to be widely researched and used. Zagoruyko et al. [25] integrate attention mechanisms by using the attention relationship matrix of the middle layer of the teacher network as knowledge to transfer it to the student network. Additionally, the Probabilistic Knowledge Transfer (PKT) method [13] models

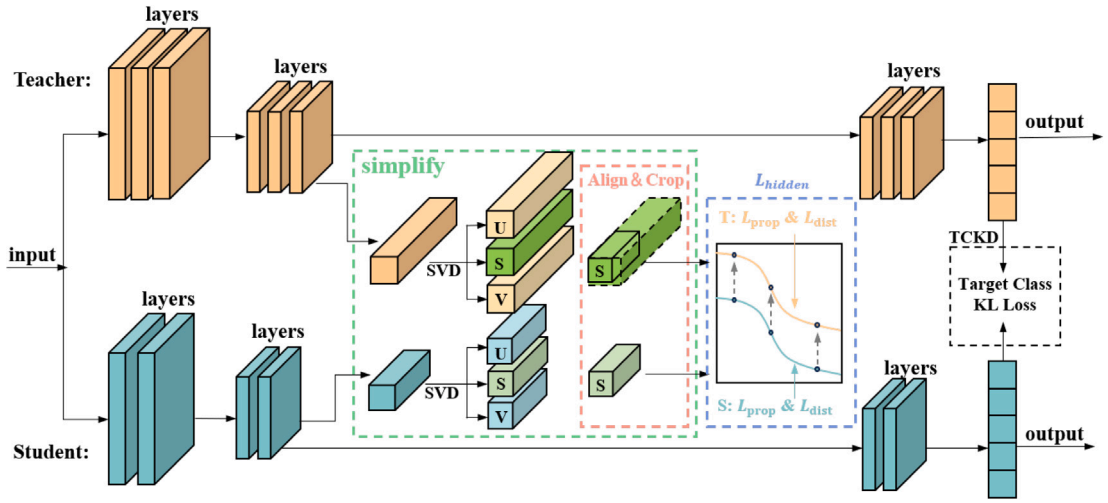


Fig. 1. The overall framework of our approach. The light orange branch represents the teacher network, and the cyan branch represents the student network. We simplify the middle hidden layer information of the teacher–student network model (green dotted box), including SVD and feature extraction. Specifically, we unify the information extracted in the middle (rose red dotted box), including the align and crop operations. The L_{hidden} loss is used to calculate the correlation of finally extracted hidden layer knowledge(light blue dotted box). Finally, we use the TCKD method to distill the information of the Target Class layer. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the relationships between middle-layer features by using probability distributions, and transfers these relationships as knowledge to the student network. The method described in [16] attempts to capture the relationships between consecutive layers as knowledge by generating the flow of the solution procedure to obtain feature map correlation gram matrices. The approach is structured KD for dense prediction [26] computes structural relationships between layers to transfer knowledge. Contrastive representation distillation [27] uses contrastive learning to transfer knowledge from the middle layer of the teacher network, while the method in Relational knowledge distillation [15] calculates cosine similarities between the middle layers of the teacher network to obtain structural relationships and then transfers this relationship matrix as knowledge to the student network. The subsequent method, distilling knowledge via knowledge review [28], builds upon this by using more complex mapping relationships to obtain information from the middle layer of the teacher network and transfer this knowledge to the student. Additionally, some methods do not directly seek a mapping relationship to obtain information from the middle layer of the teacher network. For example, The method in [9] establishes a linear pairwise difference relation knowledge loss function to transfer pairwise difference knowledge from a large teacher network to a lightweight student network.

The aforementioned methods can all be categorized as one category because they all try to obtain some kind of relationship between the middle layers of teacher and student networks and transfer the relationships as knowledge from the teacher to the student. However, the process of obtaining the relationships needs to be further refined. For instance, several methods [14] derive the relationship matrices directly through mapping high-dimensional features in the middle layer. However, this mapping process confuses the original information, and therefore increases the complication of knowledge, making it less conducive to student learning. On the other hand, several methods, such as [15,28], indirectly acquire the relationship matrix by calculating the similarity or distance of the middle-layer information. However, these methods do not effectively transfer the original information and limit the utilization of the knowledge in the teacher model, yielding suboptimal results when there is a significant difference between the teacher and student models. Some work [29] addresses the knowledge transfer problem by introducing intermediary networks. While they alleviate the difficulty of knowledge transfer to some extent, they require the separate design of a suitable intermediary network to bridge the gap between the teacher and student networks. These intermediary networks unavoidably increase time costs and result in more information loss. Furthermore, works presented in [30,31] use channel pruning to improve the efficiency of transferring the middle-layer knowledge, which adds extra steps to the knowledge extraction process.

Generally, the pruning operation works better for shallower layers in deep networks. Therefore, when transferring knowledge from a complex teacher network to a simpler student network, repetitive fine-tuning and pruning during training have to be executed and the additional complexity is unavoidably increased.

To address these shortcomings of the above-mentioned methods, we propose a unified middle-layer knowledge extraction module based on SVD. This novel module is inspired by [32], where they used the singular values in the diagonal matrix to simplify the original matrix. Our module also uses singular values to transform, align, and simplify the knowledge in the middle layer of the teacher network. Different from the above methods, our module is one-step and does not require repeated fine-tuning.

3. Methodology

3.1. Framework overview

The overall framework of our approach is shown in Fig. 1. The orange branch at the top is the teacher network, and the cyan branch at the bottom is the student network. The input of the framework is RGB images and the output is classification results. The content in the light-green-dotted box is our knowledge extraction module that extracts knowledge from the middle layer in the teacher network based on SVD, where we specify a light-orange-dotted box to show the simplifying and aligning procedure of knowledge extraction.

Through SVD operation, we simplify the intermediate feature matrix to U, S, V three simple matrices, where U, V is a pair of orthogonal matrices, and S is the singular value diagonal matrix. In particular, we have made some adjustments to the singular value matrix for the middle-layer information asymmetry (see inside the orange dotted box). We prune the middle layer of the teacher network and then align it with the student network to further transfer knowledge.

In our method, three loss functions are used to constrain the whole KD process. A novel loss function, denoted as $\mathcal{L}_{hidden}$, is used to extract the feature knowledge of the middle hidden layer. $\mathcal{L}_{KD}$ and $\mathcal{L}_{CE}$ of the target class layer are the traditional KD loss functions and classification loss functions, respectively.

3.2. SVD-based middle-layer feature extraction

3.2.1. SVD theory

According to the theorem of SVD, for any $m \times n$ matrix A, we can obtain the following symbolic representation: $A = USV^T$, where $U \in \mathbb{R}^{m \times min(m,n)}$ and $V \in \mathbb{R}^{min(m,n) \times min(m,n)}$ are orthogonal matrices, $S \in \mathbb{R}^{min(m,n) \times min(m,n)}$ is a diagonal matrix. $S = \begin{bmatrix} S_1 & 0 \\ 0 & 0 \end{bmatrix}$, $S_1 = \text{diag}\left(\lambda_1, \lambda_2, \ldots \lambda_K\right)$, $K = \text{rank(A)} \leq min(m,n)$. Each element λ_i ($i \in [1, K]$) in S_1 represents the significance or energy distribution along different directions in the original matrix. A larger value indicates that the corresponding direction holds more crucial information or contributes more energy within the original matrix. We consider that transferring the large values corresponding to crucial knowledge within the middle layers to the student network can be advantageous for enhancing the capabilities of the student network.

3.2.2. Feature matrix decomposition

We first decompose the middle-layer feature matrix $F^T \in R^{M_1 \times M_2}$ of a teacher network T by SVD. Inspired by [20,33], we can decompose T into

$$\text{SVD}(F^T) = U_t \Sigma_t V_t^{\mathbf{T}} \tag{1}$$

where $\boldsymbol{U_t} = \left[\alpha_1^T, \alpha_2^T, \ldots, \alpha_M^T\right] \in \Omega$, α^T is the eigenvector of $F^T \left(F^T\right)^{\mathbf{T}}$, $\boldsymbol{V_t} = \left[\beta_1^T, \beta_2^T, \ldots, \beta_M^T\right] \in \Omega$, β^T is the eigenvector of $\left(F^T\right)^{\mathbf{T}} F^T$, $\boldsymbol{U_t U_t^T} = \boldsymbol{V_t V_t^T} = E, E \in R^{M \times M}$ is the identity matrix, $\boldsymbol{\Sigma_t} = \left[\lambda_1^T, \lambda_2^T, \ldots, \lambda_M^T\right]$ is a diagonal matrix, M represents the $min(M_1, M_2)$.

The middle-layer feature of a student network is set as matrix $F^S \in R^{N_1 \times N_2}$, and the corresponding decomposition is

$$\text{SVD}(F^S) = U_s \Sigma_s V_s^{\mathbf{T}} \tag{2}$$

where $\boldsymbol{U_s} = \left[\alpha_1^S, \alpha_2^S, \ldots, \alpha_N^S\right] \in \Omega$, α^S is the eigenvector of $F^S \left(F^S\right)^{\mathbf{T}}$, $\boldsymbol{V_s} = \left[\beta_1^S, \beta_2^S, \ldots, \beta_N^S\right] \in \Omega$, β^S is the eigenvector of $\left(F^S\right)^{\mathbf{T}} F^S$, $\boldsymbol{U_s U_s^T} = \boldsymbol{V_s V_s^T} = E, E \in R^{N \times N}$ is the identity matrix, $\boldsymbol{\Sigma_s} = \left[\lambda_1^S, \lambda_2^S, \ldots, \lambda_M^S\right]$ is a diagonal matrix, N represents the $min(N_1, N_2)$. $\boldsymbol{U_t}, \boldsymbol{V_t}$ and $\boldsymbol{U_s}, \boldsymbol{V_s}$ belong to the same eigenspace Ω. Intuitively, it is a feature extraction process from the same input X based on the same classification criteria. According to the explanation of SVD in [33], $\boldsymbol{U_t}, \boldsymbol{V_t}$ and $\boldsymbol{U_s}, \boldsymbol{V_s}$ are both the matrices consisting of orthogonal vectors. λ^T is a rearrangement of the importance of the middle-layer features in the teacher network and can represent the characteristics of the middle-layer feature matrix. λ^S has the same effect on the student network. If the student network can fit the singular value matrix of the teacher network well, the student network can learn the arrangement of the middle-layer information of the teacher network. Therefore, knowledge transfer in the middle layer of the teacher network is simplified to feature vector arrangement, namely, the learning of the singular value matrix of the middle-layer features of the teacher network. It is worth noting that the process of the student learning knowledge from the teacher is to keep bringing λ^S closer to λ^T. In such process, $\boldsymbol{U_s}, \boldsymbol{V_s}$ is continuously adjusted, and when eventually λ^S and λ^T are about the same, the matrix $\boldsymbol{U_s}, \boldsymbol{V_s}$ will approximate to $\boldsymbol{U_t}, \boldsymbol{V_t}$. Thus, knowledge is transferred from the teacher network to the student network.

3.2.3. Knowledge transfer and learning

According to the analysis of SVD in [33], there will be many zeros in the singular value matrix after SVD. These zeros are useless information, so they can be reduced to simplify the original feature. In addition, it has been proved in [34] that the simplified decomposition matrix can still be restored approximately to the original feature matrix. Inspired by this, we have the following formula,

$$\left[\alpha_1, \alpha_2, \ldots, \alpha_u\right] \mathcal{S}_{\text{u}} \left[\beta_1, \beta_2, \ldots, \beta_u\right]^{\mathbf{T}} \cong \left[\alpha_1, \alpha_2, \ldots, \alpha_v\right] \mathcal{S}_{\text{v}} \left[\beta_1, \beta_2, \ldots, \beta_v\right]^{\mathbf{T}}, \tag{3}$$

where $v \leqslant u$, S_u is the singular value matrix after SVD. Replacing S_u by S_v in which all zeros are omitted as an approximate characteristic of the feature matrix can minimize the information loss. Therefore, we can apply this method to the knowledge extraction of middle-layer between different network architectures. For the misalignment of information in the middle layers of two kinds of teacher–student networks, the proposed method can be successfully applied and achieve good results.

3.3. Simplifying and aligning middle-layer feature

3.3.1. Simplifying and aligning using SVD

In this section, we will describe how to simplify the knowledge in the middle layer of a teacher network and align the middle-layer features between a teacher network and a student network through SVD.

For deep learning models in image classification, the feature in the middle layer is a 4D tensor, consisting of N (batch size), H (height), W (width), and C (channel).

When the structures and parameter capacities of the teacher and student networks differ greatly, the feature tensors in the middle layer must have different dimensions in all height, width and channel directions. This does not facilitate knowledge transfer as it is difficult to fit feature matrices when their dimensions are different along the same direction. Therefore, to efficiently transfer the middle-layer knowledge from the teacher to the student, their dimensions should be aligned in all directions in advance.

Fortunately, it is easy to align feature matrices in different directions by virtue of SVD. After decomposing the feature matrix of the middle layer of the teacher network, a singular value matrix S^T can be obtained. According to the last section, the learning process of the student model is to fit the S^T matrix decomposed from the middle-layer feature matrix in the teacher model. S^T is a positive semi-definite diagonal matrix and includes a large number of zero and near-zero values. The information contained in dimensions corresponding to these values is trivial. Transferring them, instead of enhancing the performance of student models, will increase the learning complexity. Thus, we reduce these dimensions by omitting these zero and near-zero values to achieve the alignment between the feature matrices of the student and teacher models.

The number of rows, columns or channels of feature matrices in the middle layers of the teacher and student networks may be different, so we develop two approaches to align the height and width, or the channel, respectively, as shown in Fig. 2.

3.3.2. Channel alignment

We first align the channels. C^T and C^S denote the channels in one middle layer of teacher and student networks, respectively. $|C^T|$ and $|C^S|$ are determined by the specific network structures of the teacher and student models, typically $|C^T| > |C^S|$. We flatten the feature matrix $F_c^T, c \in \{1, \ldots, |C^T|\}$ of each channel in the middle layer of the teacher network as a vector and stack these vectors of all channels as a matrix F^T. After applying SVD on this matrix, we obtain the singular-value matrix S^T, and each singular value corresponds to one row of F^T, i.e., the feature matrix of one channel. We sort all singular values in S^T. Then we omit $|C^T| - |C^S|$ channels which correspond to the smallest singular values which are all zeros or near-zeros Thus, the channels can be aligned for the middle-layer features in the teacher and student networks, as shown in the left of Fig. 2. Because the channels corresponding to zero or near-zero singular values contain very limited knowledge, ignoring these redundant channels will not affect the knowledge transfer from the teacher network to the student network.

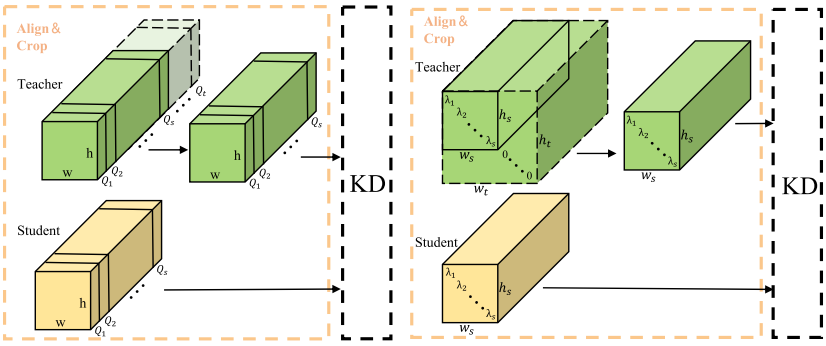


Fig. 2. Simplified and unified middle-layer information module schematic. **Left:** Unification in the channel dimension, **Right:** Unification in the H, W dimension.

3.3.3. Height and width alignment

Next, we align the height and width of the feature matrix for all retained channels. There are many zero or near-zero values in the singular-value matrix S_c^T obtained by applying SVD on the feature matrix F_c^T at the cth channel. According to the above explanation, ignoring this redundant information corresponding to those zero or near-zero-valued singular values will not result in information loss in the middle layer of the teacher network. Therefore, the middle-layer feature matrices of each channel between the teacher and student networks can be aligned by omitting rows and columns in F_c^T corresponding to the smallest $M - N$ singular values, as shown in the right of Fig. 2.

3.3.4. Comprehensive alignment strategy

Ultimately, regardless of variations in the dimension of rows, columns, and channels of the middle-layer features between teacher and student networks, we are able to simplify and align them, enabling effective knowledge transfer. In Appendix Algorithm 1, we briefly describe the specific process of our method in transmitting middle-layer information. Note that in our SVD method, there are no special requirements for N (batch size), H (height), W (width), and C (channel). Similarly, the middle-layer feature matrix F^T does not necessarily have to be a square matrix. Theoretically, they can be of any shape. The specific mathematical method of SVD can be found in [20]. The above knowledge extraction module can independently extract the important information from different rows, columns and channels so that the student network can fully learn every important knowledge. At the same time, our module can discard unnecessary information in the teacher network, reduce the complexity and improve the efficiency of the learning process of the student network. See Sections 4.3 and 4.4 for specific experimental analysis.

3.4. Loss functions

As mentioned in the last two subsections, the task that the student model learns the middle-layer knowledge from the teacher model is to make the singular values of the student model approximate those of the teacher model, which have been aligned well in the directions of the row, column and channel. Denote λ_i^T and λ_i^S as the ith singular values of the middle-layer features of the teacher and student models, respectively. The distributions of singular values of the middle-layer features in the teacher and student models are represented by h^T and h^S, respectively. Thus, the learning task in this paper can be converted to the task of approximating the distribution h^S to h^T, and approximating the numerical values of two distributions as close as possible.

The KL divergence $\mathcal{L}_{prop}$ is usually used as the constraint to approximate two distributions. However, the KL divergence can only ensure the consistency of the shapes between two distributions, which is not capable of constraining their values as close as possible, according to our following observation:

Proposition 1. *The KL divergence can only enforce shape similarity between the distributions of singular values of the middle-layer features of the teacher model and the student model; it cannot guarantee the values of the two distributions become as close as possible to each other.*

Proof. The probability of each singular value of the teacher and the student can be approximately defined as follows:

$$h_i^T = \frac{\lambda_i^T}{\sum_i \lambda_i^T}\,(i = 1, 2, \ldots), \quad h_i^S = \frac{\lambda_i^S}{\sum_i \lambda_i^S}\,(i = 1, 2, \ldots) \tag{4}$$

The loss function that constrains the two distributions is defined as $\mathcal{L}_{prop}$:

$$\mathcal{L}_{prop} = KL\left(h^S \parallel h^T\right) = \sum_i h_i^S ln \frac{h_i^S}{h_i^T} \tag{5}$$

Minimize the $\mathcal{L}_{prop}$ to make h_i^S approach h_i^T, we have

$$min\left(KL\left(h^S \parallel h^T\right)\right) = min\left(\sum_i h_i^S ln \frac{h_i^S}{h_i^T}\right) \tag{6}$$

$$\frac{\partial(KL)}{\partial h_i^S} = \left(h_i^S\right)' ln \frac{h_i^S}{h_i^T} + h_i^S \frac{h_i^T}{h_i^S} \frac{\left(h_i^S\right)'}{h_i^T} \tag{7}$$

$$\frac{\partial(KL)}{\partial h_i^S} = 1 + ln \frac{h_i^S}{h_i^T} \tag{8}$$

When $\frac{\partial(KL)}{\partial h_i^S} \to 0$, $1 + ln\frac{h_i^S}{h_i^T} \cong 0$. Thus, we can approximate $h_i^S \cong e^{-1} h_i^T$.

$$\because \quad h_i^S = \frac{\lambda_i^S}{\sum_i \lambda_i^S}, \quad h_i^T = \frac{\lambda_i^T}{\sum_i \lambda_i^T}, \tag{9}$$

$$\therefore \quad \frac{\lambda_i^S}{\sum_i \lambda_i^S} = e^{-1} \frac{\lambda_i^T}{\sum_i \lambda_i^T}, \quad \frac{\lambda_{i+1}^S}{\sum_i \lambda_i^S} = e^{-1} \frac{\lambda_{i+1}^T}{\sum_i \lambda_i^T}, \tag{10}$$

$$\therefore \quad \frac{\lambda_{i+1}^S}{\lambda_i^S} = \frac{\lambda_{i+1}^T}{\lambda_i^T}, \tag{11}$$

So we can draw that the values of λ_i^S has the equivalent consistency with λ_i^T. However,

$$\because \quad \Sigma\lambda_i^S \neq \Sigma\lambda_i^T, \quad \therefore \quad \lambda_i^S \neq \lambda_i^T. \tag{12}$$

Thus, we can draw that the specific values of λ_i^S and λ_i^T are not equal. □

Therefore, the results obtained by using only the KL loss as a constraint are not satisfactory. This is because if we want the student model to learn the teacher model as accurately as possible, we have to ensure that not only the shapes of h^S and h^T are consistent, but also their specific distribution values have to be as close to each other as possible.

In order to impose constraints on specific values while preserving the consistency between the distributions λ^S and λ^T, we introduce a

regularization term Δ to adjust the Euclidean distance between each λ_i^S and λ_i^T. Therefore, the new loss function is defined as :

$$\mathcal{L}_{hidden} = \alpha \mathcal{L}_{prop} + \beta \Delta, \tag{13}$$

where Δ is defined as:

$$\Delta = \frac{1}{N}\left(\sum_i^N \left(\lambda_i^S - \lambda_i^T\right)^2\right). \tag{14}$$

The hyperparameters α and β are used to maintain a balance between $\mathcal{L}_{prop}$ and Δ in order to ensure optimal constraint effectiveness. If Δ constraint dominates, the presence of outliers in λ_i^S may cause the fitting results to deviate from the baseline, as we cannot guarantee that all values in λ_i^S conform to the distribution, as shown in Fig. 6 (*b*). Therefore,

Proposition 2. *By using the additional regularization term Δ, the new loss $\mathcal{L}_{hidden}$ can constrain the distributions of the teacher model and the student model as closely as possible, in terms of both the shape and the value.*

Proof. We firstly prove that only using regularization term Δ cannot fit the distribution of the student model close to that of the teacher model.

Without loss of generality, suppose there is one outlier λ_i^S (the case with more outliers is similar to this one), and the rest of $\lambda_i^S (i \neq j)$ are the normal values, that is, $\lambda_i^S \cong \lambda_i^T$, $|\lambda_j^S - \lambda_j^T| \gg 0$.

$$min\left(\Delta\right) = min\left(\frac{1}{N}\left(\sum_i^N \left(\lambda_i^S - \lambda_i^T\right)^2\right)\right), \tag{15}$$

Our aim is to make λ^S approach λ^T numerically by minimizing the Euclidean distance between each pair of λ_i^S and λ_i^T. Let the error be Δ,

$$\Delta = \frac{1}{N}\left(\sum_{i\neq j}^N \left(\lambda_i^S - \lambda_i^T\right)^2 + \left(\lambda_j^S - \lambda_j^T\right)^2\right) \tag{16}$$

We denote the normal values error $\Delta_i = \frac{1}{N}\left(\lambda_i^S - \lambda_i^T\right)^2$, and the outlier values error $\Delta_j = \frac{1}{N}\left(\lambda_j^S - \lambda_j^T\right)^2$. Since $\lambda_i^S \cong \lambda_i^T$ and $|\lambda_j^S - \lambda_j^T| \gg 0$, $\Delta_i \cong 0$ and $\Delta_j \gg 0$. Therefore, the average error is dominated by the outliers: $\Delta \cong \Delta_j$. When we are minimizing the error, the new value of λ_i^S become $\lambda_i^S + \Delta$. Therefore, the new $\lambda_i^S - \lambda_i^T = \Delta_j$.

All normal λ_i^S values that are originally close to λ_i^T have been deviated by Δ_j due to the influence of the outlier. When λ_j^S is significantly large, we have the first conclusion that Δ cannot constrain the student model to fit the distribution of the teacher model well, as shown in Fig. 6(*b*).

According to Proposition 1, $\mathcal{L}_{prop}$ can constrain that the distribution of the student model has a similar shape to that of the teacher model. Therefore, when applying the $\mathcal{L}_{prop}$ to all λ^S values proportionally, the impact of outliers in λ^S can be eliminated. That is, for a pair of singular values which are the normal λ_i^S and the outlier λ_j^S, we have $\frac{\lambda_j^T}{\lambda_i^T} = \frac{\lambda_j^S}{\lambda_i^S}$, which is equal to $\lambda_j^S = \frac{\lambda_i^S}{\lambda_i^T}\lambda_j^T$.

At this point, we apply regularization Δ to normalize the value of S_λ. From $\Delta_j = \lambda_j^S - \lambda_j^T$, we can derive

$$\Delta_j = \frac{\lambda_j^T \lambda_i^S}{\lambda_i^T} - \lambda_j^T = \lambda_j^T \frac{\lambda_i^S - \lambda_i^T}{\lambda_i^T} \tag{17}$$

By substituting $\Delta_i = \lambda_i^S - \lambda_i^T$ into Eq. (17), we have $\Delta_j = \frac{\lambda_j^T}{\lambda_i^T}\Delta_i$. Because the $\lambda_i^S \cong \lambda_i^T$ and $\Delta_i \cong 0$, we can derive

$$\Delta_j = \frac{\lambda_j^T}{\lambda_i^T} \cdot \Delta_i \cong 0. \tag{18}$$

Since $\Delta_j = \lambda_j^S - \lambda_j^T$, we have $\lambda_j^S - \lambda_j^T \cong 0$, $\lambda_j^S \cong \lambda_j^T$.

Thus, all the values of λ^S, including the outlier, are now similar to their corresponding values in λ^T. □

The Proposition 2 demonstrates that, under the constraint of the regularization term Δ, $\mathcal{L}_{prop}$ can effectively reduce the difference in the singular value distributions between the student and teacher networks, progressively aligning the student network's singular value distribution with that of the teacher network. As shown in Fig. 3, subfigures a, b, and c illustrate the effects of $\mathcal{L}_{prop}$, Δ, and $\mathcal{L}_{hidden}$, respectively, on constraining the singular value distribution of the student network. We first flatten the singular value matrices, obtained via SVD on the feature matrices, into 1D tensors. The loss functions now impose constraints on the two sets of 1D tensors from the teacher and student networks.

In Fig. 3, the *x*-axis represents the index of each element in the 1D tensor, and the *y*-axis represents the corresponding singular value λ for each element. The black dots indicate the initial singular values of the student network before applying constraints. After applying constraints, we use blue dots and orange dots represent new singular values of the teacher network and the student network, respectively. The curves in the figure depict the singular value distributions: the blue curve represents the singular value distribution of the teacher network, the orange curve represents that of the student network, and the orange dashed line indicates the final trend of the student network's singular values.

- Fig. 3a illustrates the effect of using $\mathcal{L}_{prop}$ as the sole constraint. In this case, the singular value distribution of the student network (orange curve) exhibits a trend similar to that of the teacher network (blue curve) but fails to achieve ideal value alignment. Consequently, the two curves are similar but do not completely overlap.
- Fig. 3b shows the scenario after introducing the regularization term Δ. In this case, due to the presence of outliers (black dots corresponding to the *i*th index), the normal points of the student network (orange curve) deviate from the corresponding points of the teacher network (blue curve). This deviation indicates that while the regularization term Δ helps constrain the singular value distribution of the student network, it does not fully eliminate the influence of outliers.
- Fig. 3c illustrates the case when $\mathcal{L}_{hidden}$ is used as the loss function. In this case, the curve of the student network (orange dashed line) closely follows the curve of the teacher network (blue curve), maintaining a similar trend. This demonstrates that $\mathcal{L}_{hidden}$ better aligns the singular value distribution of the student network, improving the alignment effect between the student and teacher models.

Fig. 4 visualizes the effects of using only $\mathcal{L}_{prop}$ and the novel loss $\mathcal{L}_{hidden}$ with a ResNet network. The left side is the distribution curve of the middle-layer knowledge of teacher and student networks obtained by using only $\mathcal{L}_{prop}$. The teacher curve (blue) and the student curve (orange) have roughly the same trend but do not perfectly overlap, which proves that the constraint on the specific numerical values mentioned in our previous analysis is not ideal. The right side demonstrates the results of using $\mathcal{L}_{hidden}$. It can be observed that the teacher curve (blue) and the student curve (orange) approximately overlap each other, which proves that the effect of the novel loss is consistent with our analysis. In addition, we also set weights α and β for the two losses in $\mathcal{L}_{hidden}$, and explored the influence of adjusting the proportional relationship between the two parts by adjusting different values of α and β on the final classification accuracy. The related experimental analysis is introduced in Section 4.5.

In addition, we use the traditional KD loss function $\mathcal{L}_{KD}$ to evaluate the overall learning results of the student network, and constantly update the gradient to make the student network learn in the direction of the teacher network.

$$\mathcal{L}_{KD}(T, S) = KL\left(P^T \parallel P^S\right), \tag{19}$$

where P^T and P^S denote the classification probability of the teacher network and the student network, respectively.

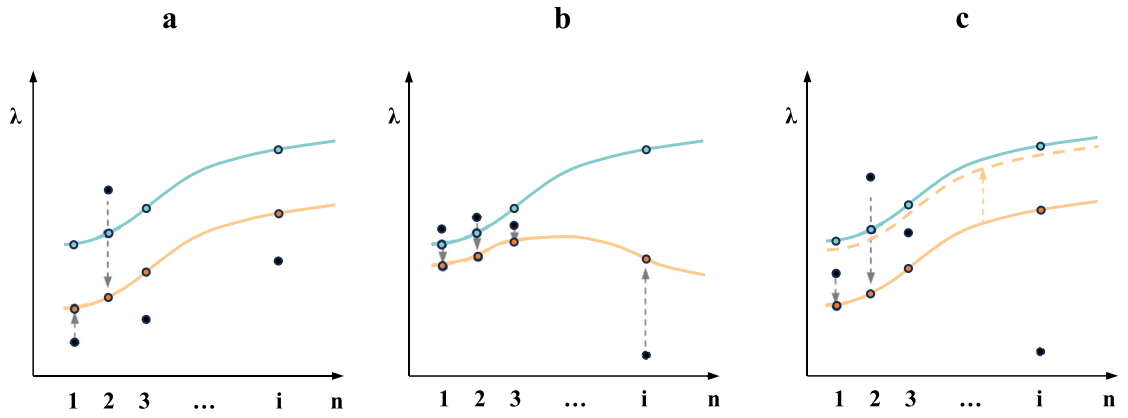


Fig. 3. The constraint effects on the student network by $\mathcal{L}_{prop}$, Δ and $\mathcal{L}_{hidden}$ Loss Functions. The horizontal axis represents the index number of the singular values, and the vertical axis represents the λ (i.e., the singular values). The dots represent singular value points. The blue and orange curves represent the singular value distributions of the teacher and student, respectively. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

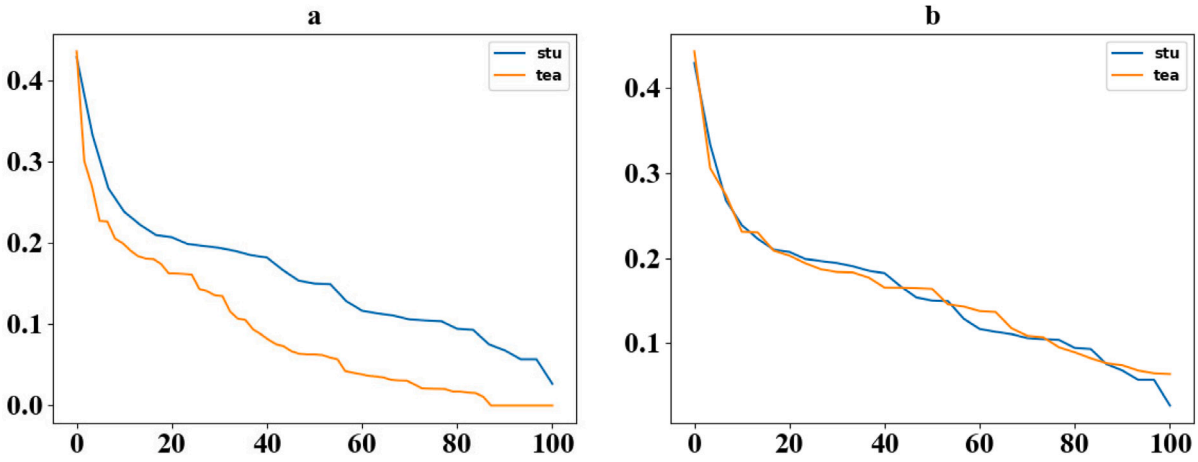


Fig. 4. The vertical axis is the λ value, the horizontal axis is the channel number, the curve represents the distribution curve of the λ value on the corresponding channel, tea represents λ^T, and stu represents λ^S. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

The cross-entropy loss, $\mathcal{L}_{CE}$, between classification probability of student network and ground-truth is employed, which is defined as follows,

$$\mathcal{L}_{CE} = \frac{1}{n}\sum_{i=1}^{n}\left(-\log P_i^{\mathrm{T}}, P_i^{\mathrm{S}}\right). \tag{20}$$

Thus, our final loss function is defined as

$$\mathcal{L}_{all} = \mathcal{L}_{KD} + \mathcal{L}_{CE} + \gamma\mathcal{L}_{hidden}, \tag{21}$$

where γ is a hyperparameter used to adjust the loss. In Section 4.5, we conducted a validation of the effectiveness of our approach.

4. Experiment

4.1. Datasets

The proposed method is tested mainly on the image classification task and evaluated on three benchmark datasets (CIFAR-10, CIFAR-100 and ImageNet 1 K).

CIFAR-10 [35] image dataset is composed of 60,000 RGB images with an image size of 32×32. The dataset is divided into 10 categories, describing animals and vehicles. Each category contains 6000 images. There are 50,000 training images and 10,000 test images.

The CIFAR-100 [35] dataset has 100 classes. Each class has 600 32×32 color images, 500 for the training set and 100 for the test set.

ImageNet 1 K [36] is composed of 1000 classes. The training set contains 1.28 million images, and the verification set contains 50k images.

4.2. Implementation details

We use the two RTX4090 GPUs on all datasets during training. Our network uses the Adam optimizer and has a learning rate of 0.001. Batch size is set to 128 on the CIFAR-10 and CIFAR-100 datasets and 32 on the ImageNet 1 K. The other parameters are set according to the settings provided in [5]. We use the same GPU number, environment configuration, and hyperparameter settings as in the original paper to reproduce the results of those comparative methods. We train 200 epochs to make the model converge. What we show in the table are the best values of our reproduced results or results from the original paper. In our SVD-KD method, the hyperparameters α and β are set to 1 and 4, respectively, and γ is set to 1. In the section on Ablation Analysis, we evaluate the influence of different values of α, β, and γ on the final classification accuracy of our method. Top-1 accuracy is used to measure the accuracy of image classification. Unless otherwise specified, for all experiments, we use the feature maps before the ReLU activation function from the last feature extraction module (specifically, the last feature extraction layer before the model's FC layer) as the middle-layer features. Additionally, we also provide links for the experimental code and configuration details on https://github.com/smartyige/SVD-KD

4.3. SVD-KD method within the same network architecture

We examine the effectiveness of our approach in distilling knowledge between the same model architectures on the CIFAR-10 dataset.

Table 1
Comparison of accuracy with state-of-the-art methods on the CIFAR-10 dataset. δ denotes the accuracy gap between various methods and the Student Model, and + denotes the improvement in accuracy.

Teacher: ResNet-34 (95.88%) Student: ResNet-10 (92.30%)								
Method	AT [25]	KD [3]	FT [37]	AT+KD [25]	DKD [5]	InDistill [30]	CTKD [38]	**SVD-KD**
Accuracy(%)	92.87	92.81	93.15	93.11	94.76	93.77	94.07	**95.45**
δ	+0.57	+0.51	+0.84	+0.81	+2.46	+1.47	+1.77	**+3.15**

Table 2
Comparison of accuracy with state-of-the-art methods on the CIFAR-100 dataset. δ denotes the accuracy gap between various methods and the student model, and + denotes the improvement in accuracy.

Teacher: ResNet-34 (80.27%) Student: ResNet-18 (72.50%)										
Method	FitNet [6]	RKD [15]	CRD [27]	OFD [39]	ReviewKD [28]	InDistill [30]	KD [3]	DKD [5]	CTKD [38]	**SVD-KD**
Accuracy(%)	73.50	73.90	75.51	74.95	75.63	76.07	73.33	76.32	75.37	**78.97**
δ	+1.00	+1.40	+3.01	+2.45	+3.13	+3.57	+0.83	+3.82	+2.87	**+6.47**

Table 3
Comparison of accuracy with state-of-the-art methods on the ImageNet 1K dataset. δ denotes the accuracy gap between various methods and the Student model, and + denotes the improvement in accuracy.

Teacher: ResNet-34 (73.31%) Student: ResNet-18 (69.75%)								
Method	AT [25]	OFD [39]	CRD [27]	ReviewKD [28]	FT [37]	DKD [5]	CTKD [38]	**SVD-KD**
Accuracy(%)	70.69	70.81	71.17	71.61	71.03	71.70	71.32	**72.71**
δ	+0.94	+1.06	+1.42	+1.86	+1.28	+1.95	+1.57	**+2.96**

The results are shown in Table 1. The validation indicator is the final accuracy of the classification task. δ represents the accuracy difference between the student network after KD and the original student network in the final classification task. We choose the ResNet-34 model as the teacher network and set the BasicBlock as [3, 4, 6, 3]. It has 21,335,972 parameters with a size of 81.39 MB. The network has excellent classification performance on the dataset and is suitable for the teacher network. We choose ResNet-10 as the student network and set the BasicBlock to [1, 1, 1, 1]. It has 4,957,092 parameters with a size of 18.91 MB. This network is the smallest network with the same structure as the teacher network, so it is more suitable to be used as the student network to verify the effect of KD under the same structure. We compare with the mainstream KD methods in recent years, including the traditional KD method (e.g. DKD) and the method with the addition of auxiliary hidden layer characteristics (e.g. AT, KD, FT, AT+KD, Indistil). In terms of classification accuracy, our method does achieve an improvement of 3.15%, which exceeds the accuracy of other methods on the same task.

We continue to validate our approach on the CIFAR-100 (with more categories) and ImageNet 1 K (with more complex and larger image sets) datasets. The experimental results are shown in Tables 2 and 3. Due to the increasing difficulty of the task, in order to fully show the influence of different distillation methods on the final classification accuracy, we choose a more suitable model. We use ResNet-34 as the teacher network and set the BasicBlock to [3, 4, 6, 3]. We use ResNet-18 as the student network and set BasicBlock as [2, 2, 2, 2]. According to the results in the table, our method has a significant improvement of 6.47% on CIFAR-100 and 2.96% on the more difficult ImageNet 1 K dataset (1000 classification). Compared with other KD methods, our method also has great improvement. Experiments show that our method is consistent in improving the effectiveness of this kind of conventional KD task.

We conduct KD experiments on the CIFAR-100 dataset by using the teacher and student networks, which have the same network model and the same middle-layer features (see Table 4). We take the features from the penultimate convolution block of ResNet (size $14 \times 14 \times 256$) and VGG network model (size $14 \times 14 \times 512$), specifically the features before the ReLU activation function, as the middle-layer features. We use two broad categories of comparison methods. The "Logic" method is the KD method for the traditional one, and the "feature" method is the KD method for additional supervision by adding middle-layer features. From the experimental results in Table 4, it can be seen that our method is more effective than most of the other methods in this KD task. Only the more complex method, ReviewKD [28], is better than our method. This is because it requires elaborate feature extraction and fusion for each layer of the network, and adds a large number of information transfer and attention mechanisms to the training process, which makes the model more complex. This method is very different from our method and other methods, all of which are a single-way KD approach with a simpler and more efficient architecture.

4.4. SVD-KD method across different network architectures

In order to verify that our method can be used for KD across models, we conducted experiments on the CIFAR-100 dataset by using the teacher and student networks with different dimensions of features in the middle layer. The experimental results are shown in Table 5. We execute three groups of comparison. The first two groups of experiments use a ResNet-34 as the teacher network, a ShuffleNet-V2 and a MobileNet-V2 as the student network, respectively. In the latter group of experiments, a VGG-13 is used as the teacher network, and a MobileNet-V2 is used as the student network. The experimental results show that we have certain competitive advantages compared with similar methods.

In addition, we found that the effect of ReviewKD method is slightly better than our method when the size gap between a teacher network and a student network is not large. However, when there is a significant size gap between the teacher and student networks, the ReviewKD method becomes ineffective, whereas our method still achieves good results(show in Table 8); furthermore, the accuracy of our method is much higher than that of ReviewKD when the teacher–student network is evaluated on more challenging datasets (ImageNet 1 K) in Table 3. When the student network is similar to the teacher network, ReviewKD enables the student network to fully absorb and utilize the middle-layer knowledge of the teacher's network. In contrast, when there is a large gap between the teacher–student network (e.g., the student network is lightweight in this experiment). For example, in Table 5, when there is a significant accuracy gap between the teacher (ResNet-34) at 80.27% and the student (MobileNet-V2) at 64.60%, our method achieves the best results. This fully demonstrates that our method can extract the most useful middle-layer knowledge of the teacher and transfer it to the student network. It is worth noting that such a small student network is easier to deploy on edge devices.

Table 4
Accuracy at different stages for teacher–student networks with the same structure on the CIFAR-100 dataset. The values in the first row represent the accuracy achieved by the original teacher–student network without distillation.

Method	Model	Acc(%)	δ	Model	Acc(%)	δ
–	Teacher(ResNet-50)	79.34	–	Teacher(VGG-13)	74.64	–
	Student(ResNet-34)	74.21	–	Student(VGG-8)	70.36	–
Feature	FitNet [6]	74.27	+0.06	FitNet [6]	71.02	+0.66
	RKD [15]	74.67	+0.46	RKD [15]	71.48	+1.12
	CRD [27]	75.36	+1.15	CRD [27]	73.94	+3.58
	OFD [39]	74.98	+0.77	OFD [39]	73.95	+3.59
	ReviewKD [28]	**79.79**	**+5.58**	ReviewKD [28]	**74.84**	**+4.48**
	InDistill-l-CLS [30]	74.87	+0.66	InDistill-l-CLS [30]	73.97	+3.61
	CTKD [38]	76.62	+2.41	CTKD [38]	73.52	+3.16
Logic	KD [3]	74.97	+0.76	KD [3]	72.98	+2.62
	DKD [5]	78.07	+3.86	DKD [5]	74.68	+4.32
	SVD-KD	79.47	+5.26	**SVD-KD**	74.69	+4.33

Table 5
Accuracy comparison of distillation effects on teacher–student networks with different structures on the CIFAR-100 dataset.

Model	Acc(%)	δ	Model	Acc(%)	δ	Model	Acc(%)	δ
Teacher (ResNet-34)	80.27	–	Teacher (VGG-13)	74.64	–	Teacher (ResNet-34)	80.27	–
Student (ShuffleNet-V2)	70.50	–	Student (MobileNet-V2)	64.60	–	Student (MobileNet-V2)	64.60	–
FitNet [6]	73.59	+3.09	FitNet [6]	64.14	−0.46	FitNet [6]	63.16	−1.44
RKD [15]	72.28	+1.78	RKD [15]	64.52	−0.08	RKD [15]	64.43	−0.17
CRD [27]	75.11	+4.61	CRD [27]	69.73	+5.13	CRD [27]	69.11	+4.51
OFD [39]	75.98	+5.48	OFD [39]	69.48	+4.88	OFD [39]	69.04	+4.44
ReviewKD [28]	**77.45**	**+6.95**	ReviewKD [28]	**70.37**	**+5.77**	ReviewKD [28]	69.89	+5.29
KD [3]	74.07	+3.57	KD [3]	67.37	+2.77	KD [3]	67.35	+2.75
DKD [5]	76.45	+5.95	DKD [5]	69.71	+5.11	DKD [5]	70.35	+5.75
CTKD [38]	74.58	+4.08	CTKD [38]	69.01	+4.41	CTKD [38]	69.01	+4.41
SVD-KD	75.97	+5.47	**SVD-KD**	69.18	+4.58	**SVD-KD**	**70.75**	**+6.15**

4.5. Ablation analysis

4.5.1. Hyperparameter sensitivity analysis

We validate the effectiveness of our method through a series of experiments, the results of which are shown in Table 6. We employ ResNet-50 as the teacher network and ResNet-18 as the student network. The validation of our proposed SVD-KD method is conducted by evaluating its accuracy in the classification task on the CIFAR-100 dataset. In the table, the KD method without utilizing middle-layer features as additional supervision is referred to as "KD". The method denoted as "SVD+$\mathcal{L}_{prop}$" incorporates the novel module of extracting middle-layer knowledge for additional supervision in KD without using the novel loss function. Meanwhile, the "SVD+$\mathcal{L}_{hidden}$" method combines the novel knowledge extraction module and the novel loss function.

From the experimental results, it is evident that the utilization of the SVD approach leads to improved classification accuracy across the board. This demonstrates the effectiveness of our knowledge extraction module for transferring knowledge from the middle layers of the teacher network. Furthermore, when replacing the loss function from "$\mathcal{L}_{prop}$" with "$\mathcal{L}_{hidden}$", a substantial increase in the accuracy is observed, approaching the accuracy of the teacher network. This showcases the effectiveness of the novel loss function. These experimental results align with the analysis presented in the previous section.

As described above, we introduce two parts to guide the student network to learn the middle-layer knowledge of the teacher network in $\mathcal{L}_{hidden}$, i.e., the $\mathcal{L}_{prop}$ and Δ in Eq. (13). However, we believe that these two parts play different roles and effects in guiding tasks. Therefore, we set different weights (α, β) for the two parts. As shown in Fig. 5, α represents the degree of influence of $\mathcal{L}_{prop}$, and β represents the degree of influence of the regularization term Δ. It can be seen that the result at (1,1) is significantly better than at (0,0), (1,0), and (0,1), indicating that the regularization term Δ plays a corrective role for $\mathcal{L}_{prop}$. This verifies our theoretical analysis in Section 3.4. Additionally,

Table 6
Results of ablation experiments using three different methods on student model. "Accuracy" refers to the classification accuracy on the CIFAR-100 dataset. δ indicates the difference in accuracy compared to the student model, and "+" signifies the improvement in accuracy.

		Accuracy(%)	δ
Model	Teacher(ResNet-50)	79.34	
	Student(ResNet-18)	72.50	
Method	KD	73.94	+1.44
	SVD+$\mathcal{L}_{prop}$	76.37	+3.87
	SVD+$\mathcal{L}_{hidden}$	78.97	+6.47

Table 7
Impact of different γ Values on SVD-KD results on the CIFAR-100 dataset. "Acc" represents the classification accuracy.

T:ResNet-34(80.27%) S:ResNet-18(72.50%)							
γ	0	0.5	0.75	1	1.5	2	3
Acc(%)	73.33	75.15	77.86	**78.97**	78.92	78.80	78.57
T:ResNet-34(80.27%) S:ShuffleNet(70.50%)							
γ	0	0.5	0.75	1	1.5	2	3
Acc(%)	74.07	74.80	75.62	**75.97**	75.89	75.81	75.70

it is evident that the corrective effect of Δ on $\mathcal{L}_{prop}$ is not always the greater the better. We believe that an excessively large corrective effect can make it more difficult for $\mathcal{L}_{prop}$ to fit properly. After experiments (see Fig. 5), we draw a basic conclusion: simultaneously using $\mathcal{L}_{prop}$ and the regularization term Δ to constrain specific singular values of the hidden layer features achieves desirable results compared to using either of them alone. However, excessively increasing the weight of the regularization term Δ can lead to a decline in performance. Therefore, we choose a ratio of 1:4 for the final values of α and β, representing the weights assigned to $\mathcal{L}_{prop}$ and the regularization term Δ, respectively.

In the Table 7, γ is a crucial hyper-parameter that influences the balance between the different components of our loss function as shown

Table 8
Classification accuracy of different knowledge distillation methods on the CIFAR-10 dataset (ResNet-18 as the teacher network and CNN-64 as the student network).

T: ResNet-18 (96.18%) Params: 42.56(MB) S: CNN-64 (68.30%) Params: 0.22(MB)									
Method	KD [3]	FitNet [6]	AT [25]	MKT [40]	PKT [13]	CRD [27]	InDistill [30]	CTKD [38]	**SVD-KD**
Accuracy(%)	73.08	72.54	73.22	69.32	73.27	71.99	74.13	71.97	**75.63**

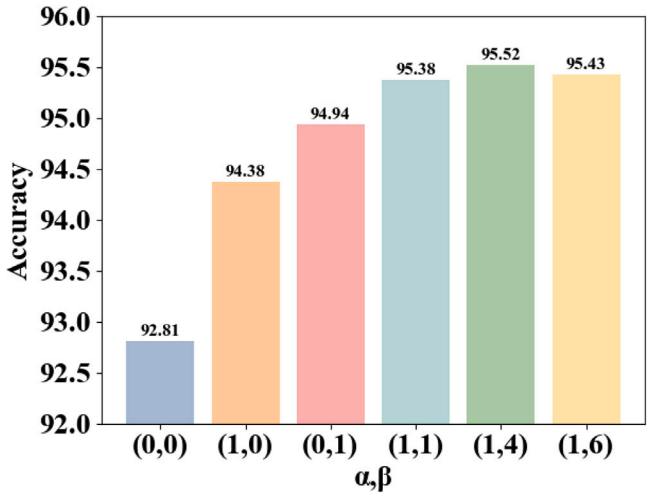


Fig. 5. Classification accuracy on CIFAR-10 dataset. α and β are the hyperparameters in $\mathcal{L}_{hidden}$, respectively. Different values of α and β represent the proportional relationship between the two components.

in $\mathcal{L}_{all} = \mathcal{L}_{KD} + \mathcal{L}_{CE} + \gamma\mathcal{L}_{hidden}$ Eq. (21). We conducted SVD-KD model compression under two teacher–student network architectures: the first using ResNet-34 as the teacher network and ResNet-18 as the student network, representing SVD-KD is used between the similar network architecture; the second using ResNet-34 as the teacher network and ShuffleNet as the student network, representing SVD-KD is used between different network architectures. As shown in Table 7, we evaluated the impact of the loss function under different γ settings on the performance of the compressed model on the CIFAR-100 dataset, using classification accuracy as the evaluation metric. It can be seen from Eq. (21) that γ represents the weight of $\mathcal{L}_{hidden}$. The results show that as γ increases from 0 to 3, the classification accuracy initially increases, reaching a maximum at $\gamma = 1$, and then slightly decreases. Therefore, when the weight γ of the loss term $\mathcal{L}_{hidden}$ is equal to 1, the model compressed based on SVD-KD achieves the best performance. Therefore, in our final SVD-KD method, γ is set to 1.

4.5.2. Visualization and analysis of middle-layer features

Similarly, in the case of the network compressed by ResNet-18 to an extremely small student network, we compare the model mentioned in this paper with the same model as in [30] and carry out a set of experiments. The CNN model is a simple network comprising 3 convolutional layers and 2 full connection layers. The convolutional layer channels for CNN-64 are set to [16, 32, 64], while for CNN-32, the convolutional layer channels are set to [8, 16, 32]. The results are shown in Table 8. It can be seen that our method has improved the classification accuracy of the CIFAR-10 dataset compared with other KD methods.

Then, we visualize the middle-layer features of different teacher–student networks with the proposed SVD-KD method to verify the effect of our method on extracting and simplifying the middle-layer knowledge of the teacher network. Since high-level semantic information is more abstract, we chose to visualize the shallow features of the network model for intuitive observation, as shown in Figs. 6 and 7.

The richness of shallow channel information is reflected in the texture richness and image information recognizability of the visualized image. The closer the image is to black, the less channel information it represents. A black image basically means that the values in that channel are almost zero. This kind of channel hardly contains any useful information to transfer knowledge to the student network. On the other hand, the channels that correspond to bright and clear images with fine textures are often the channels that contain important information. The information in these channels is the most effective knowledge to transfer to the student network.

The Fig. 6 visualizes the middle-layer features of a teacher model (left) and a student model (right) by inputting a boat image. The left of Fig. 6 shows the visualization of 64-channel middle-layer features of a ResNet-18, which is used as the teacher network. The right of Fig. 6 depicts the visualization of 32-channel middle-layer features of a student network, which consists of three convolution layers (channels are [16 32 64]) and two full connection layers.

In Fig. 6, the middle-layer features of the teacher network have a shape of [64, 56, 56]. By performing SVD decomposition along the channel dimension, we obtain U^T (left singular vectors), V^T (right singular vectors), and S^T (singular value matrix), where S^T has a size of 64. Each singular value in S^T represents the importance of information in the corresponding channel. Similarly, the middle-layer features of the student network have a shape of [32, 32, 32]. Performing SVD decomposition along the channel dimension yields U^S, V^S, and S^S, with S^S having a size of 32. To transfer knowledge, we select the most important 32 singular values from S^T of the teacher network and align them with the singular values of the student network. During this process, the student learns by adjusting U^S and V^S through backpropagation to match the corresponding U^T and V^T of the teacher network's most important singular values. This alignment ensures that the student network's middle-layer features are aligned with the teacher's along the channel dimension. The alignment along the spatial dimensions (HEIGHT and WIDTH) follows a similar process. From the visualization results, the richer middle-layer features of the teacher network result in larger singular values. In contrast, the student's middle-layer features show that the student has effectively learned all the important middle-layer features from the teacher while ignoring redundant or less useful features. This process is referred to as alignment.

The Fig. 7 visualizes the middle-layer features of the teacher and student network with similar structures. The teacher network consists of three convolution layers (channels are [16 32 64]) and two full connection layers. The student network has the same structures, with the only difference being that the channels are [8 16 32]. The left of Fig. 7 illustrates the middle-layer features of the second convolution layer of the teacher network. The right of Fig. 7 also visualizes the 16-channel middle-layer features at the second convolution layer of the student network. It can be noted that the student network exhibits rich channel information without the relatively darker channel features found in the teacher network. We believe this is because our SVD-KD method transfers important information from the teacher network to the student network for learning. Thus, the student network learns the important feature information, and the information of each channel is sufficient. From Fig. 6, we can also see that the features with rich texture details on the left are also shown on the right.

By comparing the visualization results of the middle-layer features, we can conclude that the SVD-KD method can effectively simplify and extract the middle-layer knowledge from the teacher network, both

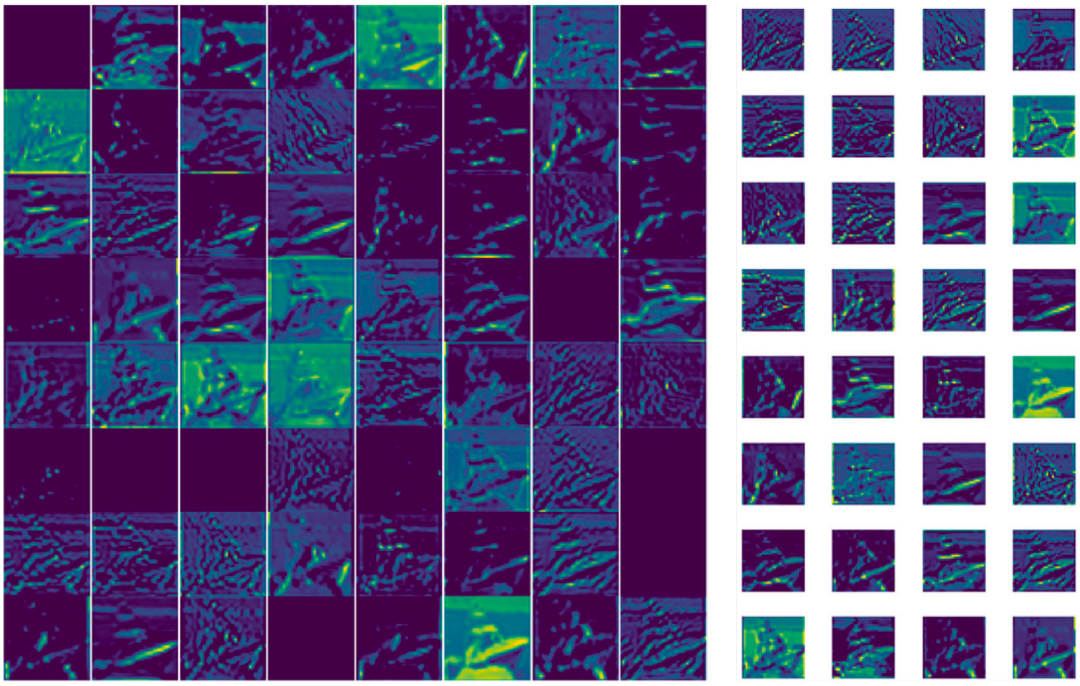


Fig. 6. Visualization of middle-layer features of the teacher and student network with different structures. **Left:** Visualization of the middle-layer features with 64 channels in a ResNet-18 teacher network (each feature size: 56 × 56). **Right:** Visualization of the middle-layer features with 32 channels extracted from a student network (each feature size: 32 × 32).

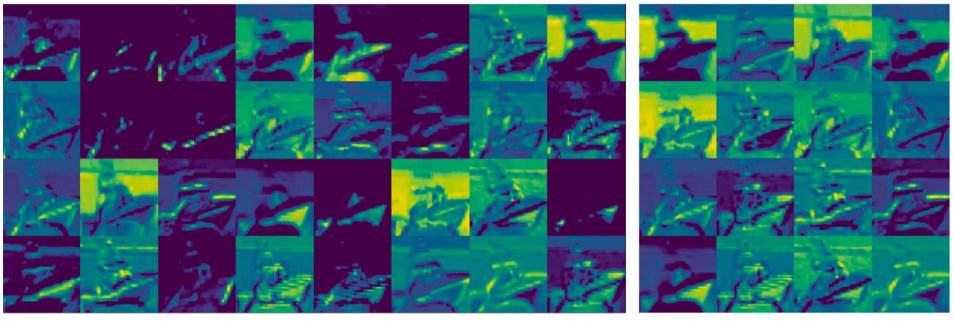


Fig. 7. Visualization of middle-layer features of the teacher and student network with similar structures. **Left:** Visualization of the middle-layer features with 32 channels extracted from a teacher network (each feature size: 32 × 32). **Right:** Visualization of the middle-layer features with 16 channels extracted from a student network (each feature size: 32 × 32).

within the same network model and across different network models. This process efficiently transfers valuable information to the student network for learning. By observing the visualization results of the middle layer of the student network, we can confirm that it is consistent with the expected effect of the SVD-KD method.

4.5.3. Time cost and complexity analysis of SVD

Finally, we further conducted a detailed experimental evaluation and analysis of the time cost and computational complexity of the SVD process, as shown in Table 9. For the time consumption aspect, we implemented the SVD computation using the SVD function available in the PyTorch library. Since the time required for this computation is hardware-dependent, we conducted experiments comparing the time consumption on different platforms, including both GPU and CPU. Regarding the computational complexity, we used the hardware-agnostic metric of FLOPs (Floating Point Operations per Second) to evaluate the SVD process. This provides a general measure of computational complexity that is independent of specific hardware configurations.

The FLOPs calculation for the feature matrix composed of batch size B, channel C, height H, and width W is as follows:

Table 9
Time cost and computational complexity of SVD on feature matrices of different sizes. The unit is seconds. RTX refers to NVIDIA series graphics cards, and CPU refers to Intel(R) Core(TM) i5-10400.

Size	RTX 2070	RTX 3060	RTX 4090	CPU	FLOPs
[32,256,7,7]	0.01620	0.01653	0.00405	0.08678	3.37×10^7
[32,512,7,7]	0.03220	0.03325	0.00795	0.16124	6.74×10^7
[32,256,14,14]	0.02360	0.02331	0.00516	0.23733	2.69×10^8
[32,512,14,14]	0.04657	0.04622	0.01026	0.46789	5.39×10^8
[32,256, 7 × 7]	0.06584	0.05101	0.03400	0.01297	1.09×10^8
[32,512, 7 × 7]	0.06745	0.05646	0.03767	0.01793	1.87×10^8
[32,256, 14 × 14]	0.23456	0.17052	0.15639	0.09179	3.19×10^9
[32,512, 14 × 14]	0.24169	0.17503	0.15878	0.11167	4.45×10^9

- matrices with the shape of [B,C,H,W] represent the matrix shape when performing SVD on the height and width dimensions.
- matrices with the shape of [B,C,H × W] represent the matrix shape when performing SVD on the channel dimension.

Refer to the computational method described in [20], assuming the matrix a has dimensions of m × n, when m≥n, the FLOPs for SVD

Table 10
Parameter and time statistics for different teacher–student models using SVD-KD and three feature-based KD methods on the CIFAR-10 dataset. All time units are in seconds. Batch size is set to 128.

	Params(MB)	F/B pass (MB)	Total (MB)	FLOPs	Train (Ours)	SVD (Ours)	ReviewKD [28]	RKD [15]	InDistill [30]
T:ResNet-50 S:ResNet-34	89.75 81.21	286.55 96.28	376.88 178.07	5.39×10^8	229.649	137.470	131.371	120.320	180.082
T:ResNet-34 S:ResNet-10	81.21 18.91	96.28 42.69	178.07 62.17	5.39×10^8	69.758	24.591	23.462	12.331	53.927
T:VGG-13 S:VGG-8	492.06 471.11	198.68 105.27	691.31 576.95	5.39×10^8	349.429	165.424	168.310	137.325	200.123
T:ResNet-34 S:ShuffleNet	81.21 4.82	96.28 47.93	178.07 53.32	1.44×10^9	108.499	59.318	58.371	50.164	107.879
T:VGG-13 S:MobileNet-V2	492.06 0.62	198.68 51.07	691.31 52.26	1.26×10^9	220.935	103.122	101.300	98.011	146.680
T:ResNet-34 S:MobileNet-V2	81.21 0.62	96.28 51.07	178.07 52.26	1.26×10^9	101.093	53.300	53.450	43.371	102.720
T:ResNet-18 S:CNN-64	42.56 0.22	62.79 0.32	106.01 0.55	3.70×10^8	26.076	8.917	–	–	54.200

is approximately $4mn^2 + 8n^3$. The term $4mn^2$ comes from the process of computing Householder transformations or tridiagonalization, while $8n^3$ comes from the QR decomposition of the tridiagonal matrix. Similarly, when m < n, the FLOPs become $4nm^2 + 8m^3$. Therefore, for a matrix of shape [B,C,H,W], the FLOPs are approximately

$$B \times C \times (4max(H,W)min(H,W)^2 + 8min(H,W)^3). \tag{22}$$

For a matrix of shape [B,C,H×W], the FLOPs are approximately

$$B \times (4max(C, H \times W)min(C, H \times W)^2 + 8min(C, H \times W)^3). \tag{23}$$

In our SVD module, we use the SVD function from the PyTorch package to calculate the singular values of individual matrices. The specific implementation details and optimizations may vary with different version and hardware, we only provide an approximate order of magnitude of FLOPs based on the general steps of singular value decomposition.

As shown in Table 9, we selected the most common shapes from the middle-layer feature matrices of the features class method. We conducted experiments on their complexity and time overhead on different hardware. When computing the SVD of matrices like [32,512,14,14], due to the large number of computations (32 × 512) but the small size of each SVD matrix (14,14), the GPU's speed ratio is faster than the CPU, demonstrating the parallel computing capabilities of CUDA on the GPU. Conversely, when computing the SVD of matrices like [32,512,14 × 14], due to the relatively small number of computations (32) but the large size of each SVD matrix (512,14 × 14), the CPU's computation speed is faster than the GPU, demonstrating the CPU's superior computing power.

Next, we further supplemented the experiment and analysis of SVD operations for the proposed method and other comparative methods under different teacher–student network architectures. As shown in Table 10, Params represents the total memory required for the model's parameters (weights and biases), measured in megabytes (MB). F/B pass indicates the maximum memory required during the forward and backward propagation processes of the model, also measured in megabytes (MB). This includes the storage of activations, gradients, and middle calculation results. Since the teacher model does not record gradients during the training process and is only used to impart knowledge to the student, the difference in F/B pass reflects, to some extent, the difference in knowledge transfer. Total refers to the total memory required to run the model, measured in megabytes (MB). FLOPs denotes the overall computational complexity of the SVD module. Train indicates the time taken to train an entire teacher–student model for one epoch, measured in seconds(s). SVD represents the time spent by the SVD module during the training process, also measured in seconds(s). However, considering that there are also significant differences in knowledge between models of different structures, the efficiency of using the SVD module in the specific knowledge distillation process should be analyzed comprehensively based on the actual situation.

In Table 10, we compare the time consumption of three representative methods for processing middle-layer features. It can be seen that our method's time consumption is comparable to ReviewKD [28], slightly less efficient than RKD [15], but significantly more efficient than InDistill [30]. Our method takes more time than the one-step alignment method like RKD [15]. However, while it is slightly less efficient in terms of time consumption compared to ReviewKD [28], which aligns numerous middle-layer matrices, our method is relatively faster than InDistill [30], which requires pruning before addressing alignment and then performing KD. Although our time consumption is not as favorable as the first two methods, our approach can handle KD when there is a significant gap between the teacher and student models (as shown in the last row of the table), which the other two methods cannot manage. Therefore, despite the increased time consumption during training, our method enhances the flexibility of processing middle-layer features.

5. Conclusion

In this paper, we employ the SVD for effective extraction and simplification of middle-layer knowledge of the teacher network and design a novel loss function to efficiently transfer them to the student network. The proposed method can be approximated as a pruning operation as it can cut the channels to remove redundancy and align them but has more efficiency. Our approach can be flexibly applied to many situations, even when the difference in structures between the teacher–student network models is very large, and even there is a misalignment of the middle layers of two networks.

Our method, despite requiring more training time for SVD of middle matrices, achieves superior KD results without impacting inference time. Extensive testing on CIFAR-10, CIFAR-100, and ImageNet 1 K datasets demonstrates its robustness and significant improvement in classification accuracy. Our approach enhances KD performance across various deep learning models and architectures. Specifically, using MobileNet as the student model, we achieved a compact 0.62 MB model suitable for edge computing platforms. Additionally, our designed CNN-64 model (0.22 MB) has an inference speed of just 19 ms (52.63 fps), meeting real-time requirements on low-resource hardware. These results highlight our method's capability to deliver high performance in constrained environments, making it ideal for embedded systems.

Our method has broad application prospects in areas such as multimodal feature alignment in multimodal tasks and extracting useful knowledge from temporal models. These will be the focus of our future work.

CRediT authorship contribution statement

Jianhua Zhang: Writing – review & editing, Writing – original draft, Supervision, Project administration. **Yi Gao:** Writing – original draft, Validation, Methodology, Data curation. **Mian Zhou:** Writing – original draft, Methodology, Formal analysis. **Ruyu Liu:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition. **Xu Cheng:** Writing – original draft, Funding acquisition, Formal analysis. **Saša V. Nikolić:** Writing – review & editing, Methodology, Conceptualization. **Shengyong Chen:** Writing – review & editing, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work was in part supported by the National Natural Science Foundation of China (62020106004, 92048301, 62202137, 62306212), Zhejiang Provincial Natural Science Foundation of China under Grant LMS25F020009, the Marie Skłodowska-Curie Postdoctoral Individual Fellowship under Grant No. 101154277.

Appendix A. Supplementary data

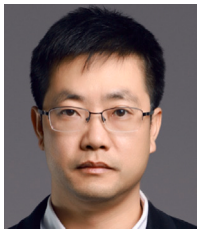
Supplementary material related to this article can be found online at https://doi.org/10.1016/j.patcog.2025.111721.

Data availability

The CIFAR-10, CIFAR-100, and ImageNet datasets, utilized in this paper for the development and validation of our method, are publicly accessible. These data are open to anyone and can be acquired through https://www.cs.toronto.edu/~kriz/cifar.html and https://image-net.org/index.php.

References

[1] Y. Cai, S. Bi, G. Li, H.P. Hildre, H. Zhang, From natural complexity to biomimetic simplification: The realization of bionic fish inspired by the cownose ray, IEEE Robot. Autom. Mag. 26 (3) (2019) 27–38.
[2] A.L. Ellefsen, S. Ushakov, V. Æsøy, H. Zhang, Validation of data-driven labeling approaches using a novel deep network structure for remaining useful life predictions, IEEE Access 7 (2019) 71563–71575.
[3] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, 2015, arXiv preprint arXiv:1503.02531.
[4] R. Müller, S. Kornblith, G.E. Hinton, When does label smoothing help? in: Advances in Neural Information Processing Systems, Adv. Neural Inf. Process. Syst. 32 (2019).
[5] B. Zhao, Q. Cui, R. Song, Y. Qiu, J. Liang, Decoupled knowledge distillation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2022, pp. 11953–11962.
[6] A. Romero, N. Ballas, S.E. Kahou, A. Chassang, C. Gatta, Y. Bengio, Fitnets: Hints for thin deep nets, 2014, arXiv preprint arXiv:1412.6550.
[7] J. Park, A. No, Prune your model before distill it, in: European Conference on Computer Vision, ECCV, Springer, Springer Nature Switzerland, 2022, pp. 120–136.
[8] M. Liang, S. Huang, W. Liu, Dynamic semantic structure distillation for low-resolution fine-grained recognition, Pattern Recognit. 148 (2024) 110216.
[9] Y. Xie, H. Wu, Y. Lin, J. Zhu, H. Zeng, Pairwise difference relational distillation for object re-identification, Pattern Recognit. 152 (2024) 110455.
[10] Y. Xie, H. Wu, J. Zhu, H. Zeng, Distillation embedded absorbable pruning for fast object re-identification, Pattern Recognit. 152 (2024) 110437.
[11] J. Gou, B. Yu, S.J. Maybank, D. Tao, Knowledge distillation: A survey, Int. J. Comput. Vis. 129 (2021) 1789–1819.
[12] P. Zhao, L. Xie, J. Wang, Y. Zhang, Q. Tian, Progressive privileged knowledge distillation for online action detection, Pattern Recognit. 129 (2022) 108741.
[13] N. Passalis, A. Tefas, Learning deep representations with probabilistic knowledge transfer, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 268–284.
[14] B. Peng, X. Jin, J. Liu, D. Li, Y. Wu, Y. Liu, S. Zhou, Z. Zhang, Correlation congruence for knowledge distillation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, ICCV, 2019, pp. 5007–5016.
[15] W. Park, D. Kim, Y. Lu, M. Cho, Relational knowledge distillation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2019, pp. 3967–3976.
[16] J. Yim, D. Joo, J. Bae, J. Kim, A gift from knowledge distillation: Fast optimization, network minimization and transfer learning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR, 2017, pp. 4133–4141.
[17] Y. Cho, G. Ham, J.-H. Lee, D. Kim, Ambiguity-aware robust teacher (ART): Enhanced self-knowledge distillation framework with pruned teacher network, Pattern Recognit. 140 (2023) 109541.
[18] S. Zhu, R. Shang, B. Yuan, W. Zhang, W. Li, Y. Li, L. Jiao, DynamicKD: An effective knowledge distillation via dynamic entropy correction-based distillation for gap optimizing, Pattern Recognit. 153 (2024) 110545.
[19] S.I. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, H. Ghasemzadeh, Improved knowledge distillation via teacher assistant, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, 2020, pp. 5191–5198.
[20] D. Kalman, A singularly valuable decomposition: the SVD of a matrix, College Math. J. 27 (1) (1996) 2–23.
[21] S.H. Lee, D.H. Kim, B.C. Song, Self-supervised knowledge distillation using singular value decomposition, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 335–350.
[22] X. Deng, Z. Zhang, Comprehensive knowledge distillation with causal intervention, Adv. Neural Inf. Process. Syst. 34 (2021) 22158–22170.
[23] Z. Meng, J. Li, Y. Zhao, Y. Gong, Conditional teacher-student learning, in: ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE, 2019, pp. 6445–6449.
[24] R. Shao, W. Zhang, J. Wang, Conditional pseudo-supervised contrast for data-free knowledge distillation, Pattern Recognit. 143 (2023) 109781.
[25] N. Komodakis, S. Zagoruyko, Paying more attention to attention: improving the performance of convolutional neural networks via attention transfer, in: ICLR, Paris, France, 2017.
[26] C. Shu, Y. Liu, J. Gao, Z. Yan, C. Shen, Channel-wise knowledge distillation for dense prediction, 2021, pp. 5311–5320.
[27] Y. Tian, D. Krishnan, P. Isola, Contrastive representation distillation, 2019, arXiv preprint arXiv:1910.10699.
[28] P. Chen, S. Liu, H. Zhao, J. Jia, Distilling knowledge via knowledge review, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2021, pp. 5008–5017.
[29] H. Zhang, Z. Hu, W. Qin, M. Xu, M. Wang, Adversarial co-distillation learning for image recognition, Pattern Recognit. 111 (2021) 107659.
[30] I. Sarridis, C. Koutlis, S. Papadopoulos, I. Kompatsiaris, InDistill: Transferring knowledge from pruned intermediate layers, 2022, arXiv preprint arXiv:2205.10003.
[31] G. Xu, Z. Liu, C.C. Loy, Computation-efficient knowledge distillation via uncertainty-aware mixup, Pattern Recognit. 138 (2023) 109338.
[32] K. Batselier, W. Yu, L. Daniel, N. Wong, Computing low-rank approximations of large-scale matrices with the tensor network randomized SVD, SIAM J. Matrix Anal. Appl. 39 (3) (2018) 1221–1244.
[33] S. Abu-El-Haija, H. Mostafa, M. Nassar, V. Crespi, G. Ver Steeg, A. Galstyan, Implicit SVD for graph representation learning, Adv. Neural Inf. Process. Syst. 34 (2021) 8419–8431.
[34] L. Liebenwein, A. Maalouf, D. Feldman, D. Rus, Compressing neural networks: Towards determining the optimal layer-wise decomposition, Adv. Neural Inf. Process. Syst. 34 (2021) 5328–5344.
[35] A. Krizhevsky, G. Hinton, et al., Learning multiple layers of features from tiny images, 2009.
[36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al., Imagenet large scale visual recognition challenge, Int. J. Comput. Vis. 115 (3) (2015) 211–252.
[37] J. Kim, S. Park, N. Kwak, Paraphrasing complex network: Network compression via factor transfer, Adv. Neural Inf. Process. Syst. 31 (2018).
[38] Z. Li, X. Li, L. Yang, B. Zhao, R. Song, L. Luo, J. Li, J. Yang, Curriculum temperature for knowledge distillation, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 1504–1512.
[39] B. Heo, M. Lee, S. Yun, J.Y. Choi, Knowledge transfer via distillation of activation boundaries formed by hidden neurons, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 3779–3787.
[40] L. Yu, V.O. Yazici, X. Liu, J.v.d. Weijer, Y. Cheng, A. Ramisa, Learning metrics from teachers: Compact networks for image embedding, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2019, pp. 2907–2916.



Jianhua Zhang Jianhua Zhang (M' 11-SM' 19) received the Ph.D. degree from University of Hamburg, Hamburg, Germany in 2012. He is currently a professor with the School of Computer Science and Engineering, Tianjin University of Technology, Tianjin, China. His current research interests include SLAM, 3D vision, reinforcement learning, and machine vision.



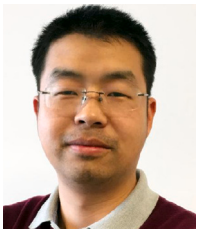
Yi Gao (Student Member, IEEE) received the B.S. degree in Integrated Circuit Design and Manufacturing from the School of Electronic Information Engineering, Tianjin University of Technology in 2015. He is currently studying for his Ph.D. in Engineering at Tianjin University of Technology. His current research interests include Deep Learning, 3D vision, reinforcement learning, and machine vision.



Mian Zhou is a Senior Associate Professor at the School of AI and Advanced Computing, XJTLU Entrepreneur College (Taicang), Xi'an JiaoTong-Livepool University, Suzhou, China. With a Ph.D. in Computer Science from the University of Reading, UK, and a Master's degree from Queen Mary, University of London, he specializes in computer vision for autonomous driving and unmanned aerial vehicles. As the former Chief Scientist at CalmCar Automotive Technology, he led the company's transformation into a leading intelligent driving system and mobility service provider. He actively participates in professional communities, including ACM, IEEE, CCF, and CSOE.



Ruyu Liu received Ph.D. degree in Control Science and Engineering from Zhejiang University of Technology, China, in 2021. Now she works with the School of Information Science and Technology, Hangzhou Normal University. Her research interests include 3D vision and simultaneous localization and mapping (SLAM).



Xu Cheng received his Ph.D. degree in Engineering from the Department of Ocean Operations and Civil Engineering, Intelligent Systems Laboratory, Norwegian University of Science and Technology (NTNU), Ålesund, Norway, in June 2020. From June 2020 to March 2022, he worked as a Postdoctoral fellow, and researcher at the Department of Manufacturing and Civil Engineering, Gjøvik, Norway. From April 2022 to June 2023, he worked at Smart Innovation Norway as a permanent researcher. Now, he is a full professor at the Tianjin University of Technology. He has applied for and coordinated more than 5 projects supported by the Norwegian Research Council (NFR), the EU, and industry. He has published more than 60 papers as first and co-author in his research interests, including data analysis and artificial intelligence in maritime operations, time series analysis, and predictive maintenance of wind turbines.



Saša V. Nikolić In 2004, he received his Ph.D. degree from the Faculty of Electronic Engineering, University of Niš, Niš, Serbia. He is currently a full professor at the Faculty of Electronic Engineering, University of Niš. Currently, his research interests include electronic information, circuits, signal processing, image processing and computer vision.



Shengyong Chen (Senior Member, IEEE) received the Ph.D. degree in computer vision from the City University of Hong Kong, Hong Kong, in 2003.,From 2006 to 2007, he received a fellowship from the Alexander von Humboldt Foundation of Germany and worked at the University of Hamburg, Hamburg, Germany. From 2008 to 2012, he worked as a Visiting Professor at Imperial College London, London, U.K., and the University of Cambridge, Cambridge, U.K. He is currently a Full Professor at the Tianjin University of Technology, Tianjin, China, and the Director of the Engineering Research Center of Learning-Based Intelligent Systems (Ministry of Education). He has been conducting research on vision sensors for robotics for more than 20 years. He has published over 300 scientific papers in international journals and conferences, including 80 articles in IEEE Transactions. He also published more than ten books in the past years and applied for more than 100 patents.,Dr. Chen received the National Outstanding Youth Foundation Award of NSFC. He organized about 20 international conferences and serves as an Associate Editor of three international journals, e.g., IEEE Transactions on Cybernetics.